

**Problem 1.****Airport data:**(a) What is the  $\alpha$  if the distribution is power law?

$\alpha = 1.612$ , with  $x_{\min} = 1$  route. Maximum-likelihood estimate  $\hat{\alpha} = 1 + n / \sum_i \ln(x_i / x_{\min})$ ,  $x_{\min} = \min_i x_i$ ,  $n = 3409$ .

|                                                |                      |
|------------------------------------------------|----------------------|
| $n = 3409$                                     | number of airports   |
| $x_{\min} = \min_i x_i = 1$                    | smallest route count |
| $S = \sum_{i=1}^{3409} \ln(x_i / 1) = 5569.43$ | sum of log ratios    |
| $n / S = 3409 / 5569.43 = 0.6121$              |                      |
| $\hat{\alpha} = 1 + 0.6121 = 1.612$            | estimate             |

(b) What is the  $\lambda$  if the distribution is exponential?

$\lambda = 0.0504$ . MLE  $\hat{\lambda} = 1 / \bar{x} = 1 / 19.845$ .

|                                       |              |
|---------------------------------------|--------------|
| $\sum_i x_i = 67652$                  | total routes |
| $\bar{x} = 67652 / 3409 = 19.845$     | mean         |
| $\hat{\lambda} = 1 / 19.845 = 0.0504$ | estimate     |

(c) What is  $[a, b]$  if the distribution is uniform?

$[a, b] = [1, 915]$ , the minimum and maximum route counts (MLE).

|                              |               |
|------------------------------|---------------|
| $\hat{a} = \min_i x_i = 1$   | fewest routes |
| $\hat{b} = \max_i x_i = 915$ | most routes   |

(d) What is  $\mu$  and  $\sigma$  if the distribution is normal?

$\mu = 19.845$ ,  $\sigma = 53.506$  (sample mean and sample standard deviation with  $n - 1$ ).

|                                           |                                 |
|-------------------------------------------|---------------------------------|
| $\hat{\mu} = \bar{x} = 19.845$            | mean, as in the exponential fit |
| $\sum_i (x_i - \bar{x})^2 = 9,756,906.22$ | sum of squared deviations       |
| $9,756,906.22 / (3409 - 1) = 2862.94$     | sample variance                 |
| $\hat{\sigma} = \sqrt{2862.94} = 53.506$  | standard deviation              |

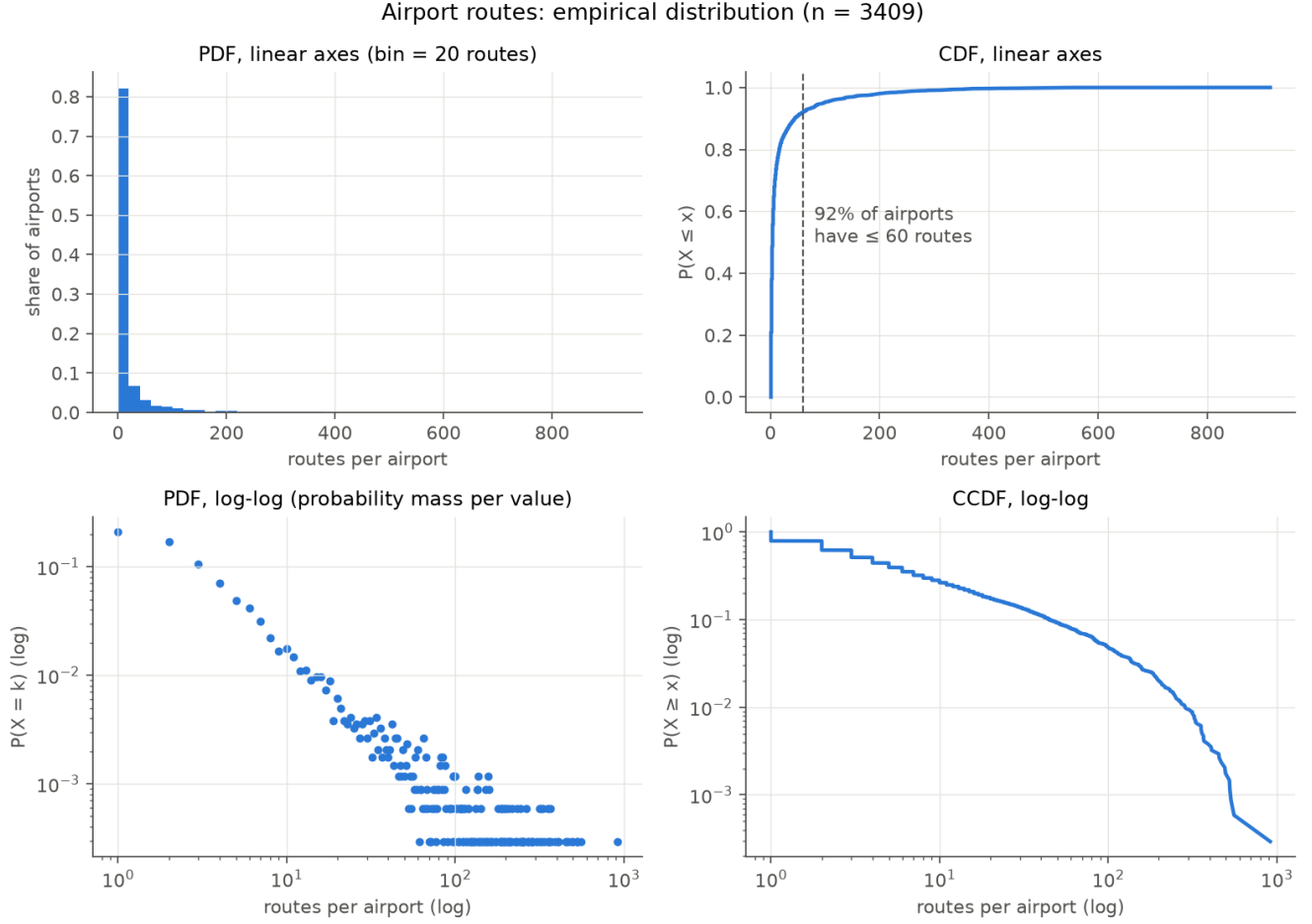
(These can be CDF, PDF, or QQplot)

For (e) to (i):

- **Samples.**  $n = 3409$  random values drawn from each fitted model (seed 6330).
- **Colours.** Data in blue, model sample in orange.

- *D.* Two-sample Kolmogorov–Smirnov distance: the largest vertical gap between the two CDFs (0 = identical).

(e) Screen shot of the visualization of distribution of data



**Figure 1:** Airport routes: empirical distribution ( $n = 3409$ ). Top row on linear axes, bottom row on log-log axes.

**PDF, linear (top left).** One tall bar at the left edge: 82% of airports fall in the first bin (fewer than 20 routes), and the bars beyond 200 are almost flat on the floor.

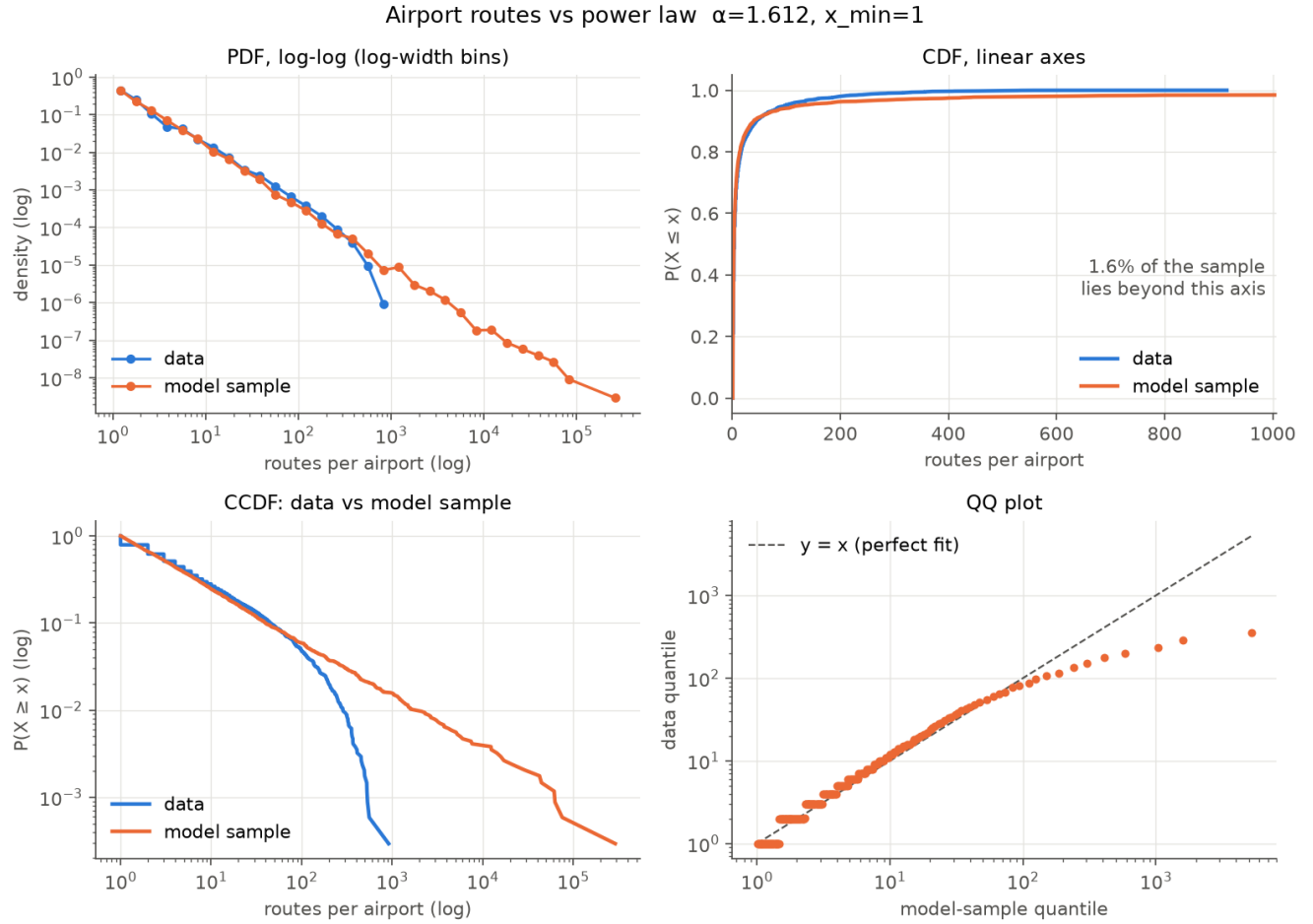
**CDF, linear (top right).** The curve rises almost vertically and reaches 0.92 at 60 routes (dashed line), then stays flat up to the maximum of 915.

**PDF, log-log (bottom left).** Share of airports with exactly  $k$  routes. The points fall along a roughly straight line from  $k = 1$  (21%) downward. In the tail many values belong to a single airport, so the points flatten into a floor at  $1/3409$ .

**CCDF, log-log (bottom right).** Share of airports with at least  $x$  routes. It is close to a straight line up to about 100 routes, then bends down and ends at the busiest airport (915 routes).

**Summary.** The median airport has 4 routes and 38% have only 1 or 2, while the busiest has 915. The mean (19.8) is five times the median and the skewness is 6.0, so a few hubs hold most of the routes.

(f) Screen shot of the visualization of distribution of data if it was power law



**Figure 2:** Airport routes vs. power law ( $\alpha = 1.612$ ,  $x_{\min} = 1$ ).  $D = 0.209$ .

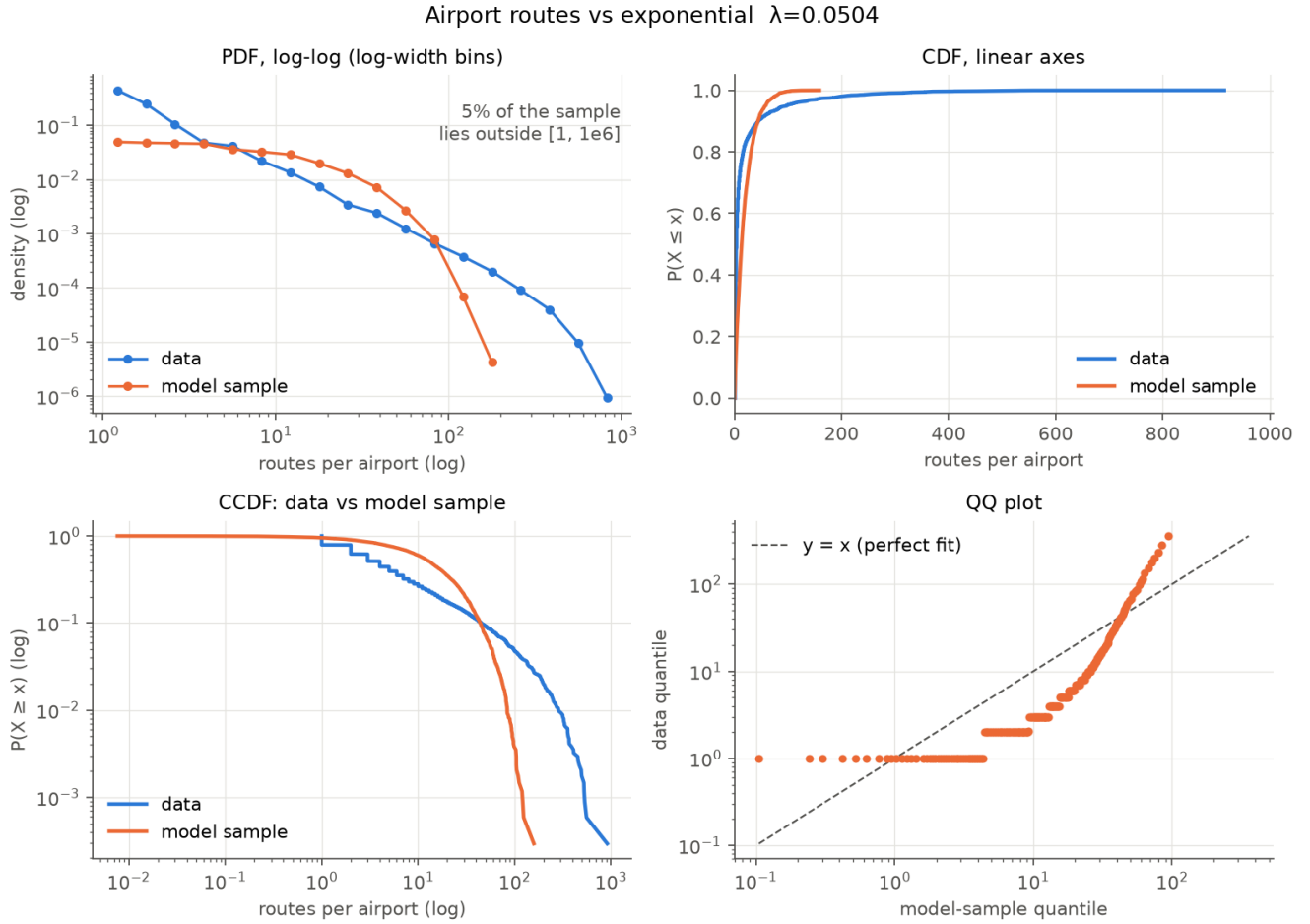
**PDF (top left).** The two lines follow the same straight slope up to about 300 routes. Above that the data drops and ends near 1000, while the model continues past  $10^5$  (its largest sampled airport has 291,593 routes).

**CDF (top right).** On linear axes the two curves almost overlap. The difference is hidden in the 1.6% of the sample that lies beyond 1000 routes.

**CCDF (bottom left).** Both follow the same straight line on log-log axes up to about 100 routes. Above that the data bends down and stops at 915; the model stays straight.

**QQ plot (bottom right).** The points lie on  $y = x$  up to about 60 routes, then run off to the right, below the line: at the 99.5th percentile the model has 5,234 routes and the data 358.

(g) Screen shot of the visualization of distribution of data if it was exponential



**Figure 3:** Airport routes vs. exponential ( $\lambda = 0.0504$ ).  $D = 0.379$ .

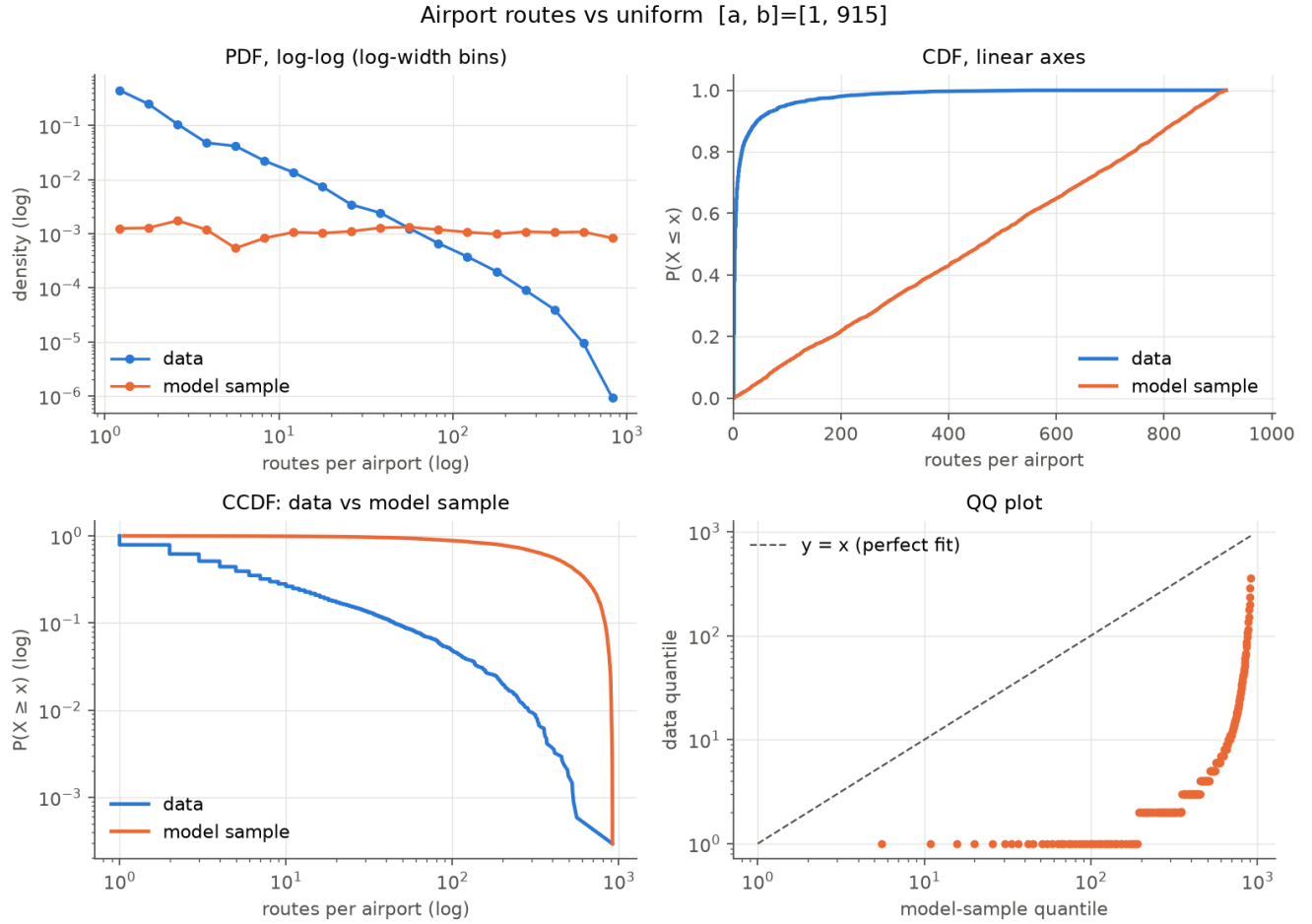
**PDF (top left).** The model is almost flat for small airports and then falls off sharply after about 50 routes. The data is the opposite: steep at the start and a long, slow tail. 5% of the sample lies below 1 route, which no airport has.

**CDF (top right).** The model reaches 1 by about 160 routes (its largest sampled airport has 159), while the data keeps rising slowly up to 915.

**CCDF (bottom left).** The model bends down early and ends near 160 routes; the data goes on to 915. With  $\lambda = 0.05$  an airport with 500 routes has probability about  $e^{-25}$ , yet the data has five such airports.

**QQ plot (bottom right).** The points are too steep: below about 40 routes they lie below  $y = x$  (the model gives small airports too many routes), and above it they rise above the line (hubs get too few).

(h) Screen shot of the visualization of distribution of data if it was uniform



**Figure 4:** Airport routes vs. uniform on  $[1, 915]$ .  $D = 0.857$ .

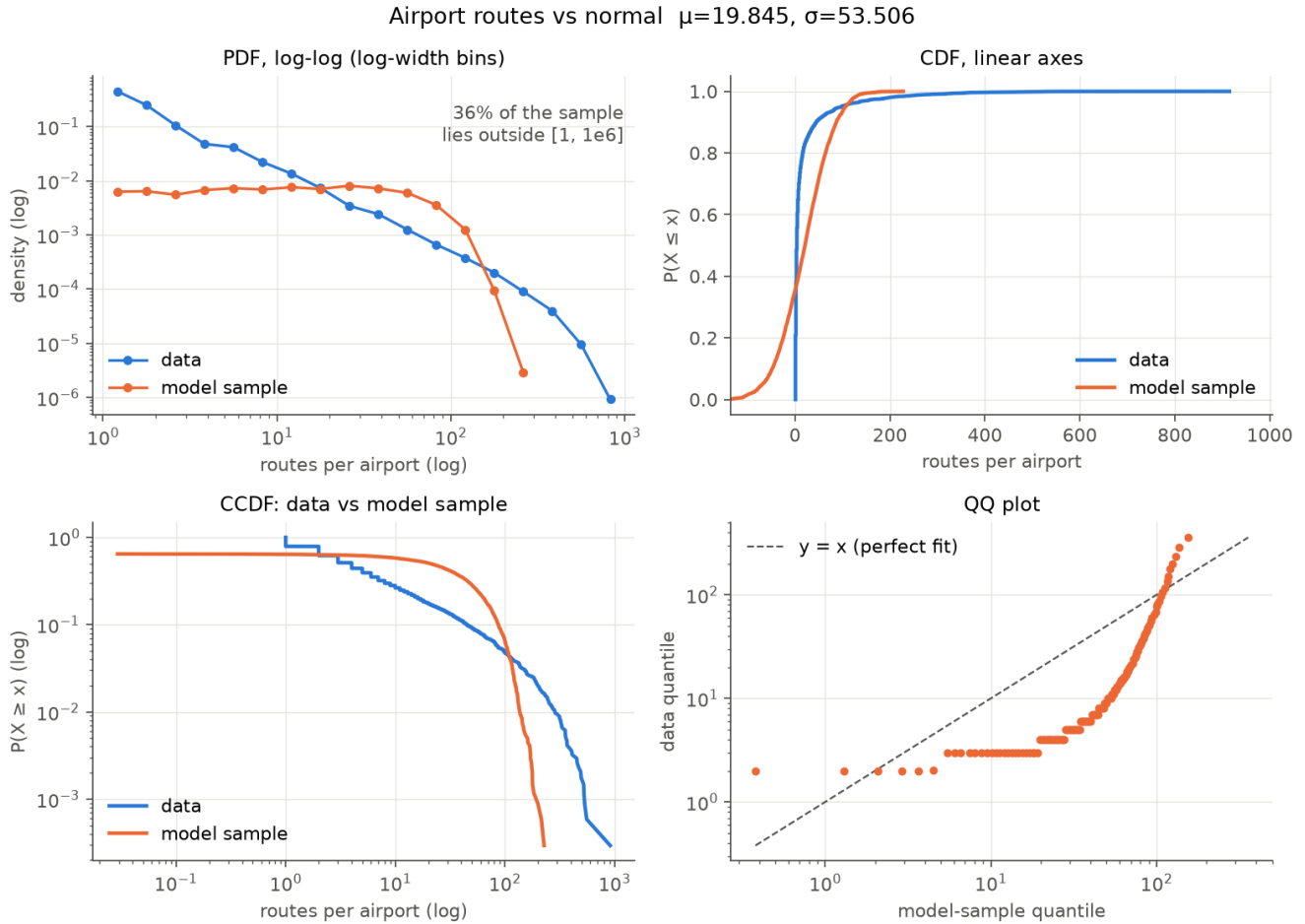
**PDF (top left).** The model is a flat line near  $10^{-3}$  across the whole range; the data falls by five orders of magnitude.

**CDF (top right).** The model is a straight diagonal from 1 to 915; the data reaches 0.92 by 60 routes. The largest gap between the two,  $D = 0.857$ , is at 60 routes.

**CCDF (bottom left).** The model stays near 1 until the right edge and then drops vertically at 915.

**QQ plot (bottom right).** Every point lies far below  $y = x$ , to the right. A typical sampled airport has about 460 routes, while the real median is 4 and 99% of real airports have fewer than 285.

(i) Screen shot of the visualization of distribution of data if it was normal



**Figure 5:** Airport routes vs. normal ( $\mu = 19.8, \sigma = 53.5$ ).  $D = 0.358$ .

**PDF (top left).** The model is flat over small airports and then drops steeply after about 100 routes. 35% of its sample is below zero, which is impossible for a route count; the log axes cannot show that part.

**CDF (top right).** The model starts below zero (negative route counts) and is S-shaped around the mean; it reaches 1 near 230 routes while the data goes to 915.

**CCDF (bottom left).** Below about 100 routes the model sits above the data, and its tail ends near 230 routes, far short of 915.

**QQ plot (bottom right).** The points are too steep, as for the exponential: small airports get too many routes and hubs too few. The tail is thin because a normal decays like  $e^{-x^2}$ .

(j) What is the distribution of data based on your observation? Discuss

|                                        | power law    | exponential | uniform | normal |
|----------------------------------------|--------------|-------------|---------|--------|
| KS distance $D$ (data vs model sample) | <b>0.209</b> | 0.379       | 0.857   | 0.358  |

**The airport data follows a power law, with a cutoff in the far tail.** It is the only one of the four models whose CCDF is a straight line on log-log axes. The data's CCDF is close to straight for more than two decades (1 to about 100 routes), and the power-law sample matches it there. The other three models fail for structural reasons, not because a parameter is slightly off:

- **Uniform** says every route count from 1 to 915 is equally likely, but most airports are tiny.
- **Normal** is symmetric and allows negative counts, while the data is bounded below at 1 and skewed to the right (skewness 6.0).
- **Exponential** has the right shape (many small, few large), but its tail falls off too fast. With  $\lambda = 0.05$ , an airport with 500 routes has probability about  $e^{-25}$ , yet the data has five airports with more than 500.

The power law is not a perfect fit.  $\hat{\alpha} = 1.61$  is below 2, so the fitted model has an infinite mean and its samples contain airports with  $10^5$  routes. Real airports cannot grow that large: there are only 3409 airports to fly to, and runways, gates and slots limit the busiest hubs. That is why the data's CCDF bends down above about 100 routes. The best description is a power law with an upper cutoff, which is typical of networks that grow by preferential attachment: new routes go mostly to airports that already have many routes.

## Movie Rating data:

(a) What is the  $\alpha$  if the distribution is power law?

$\alpha = 1.851$ , with  $x_{\min} = 1.9$  (the lowest average vote). Same MLE as for the airports,  $n = 4392$ .

|                                                |                     |
|------------------------------------------------|---------------------|
| $n = 4392$                                     | number of movies    |
| $x_{\min} = \min_i x_i = 1.9$                  | lowest average vote |
| $S = \sum_{i=1}^{4392} \ln(x_i/1.9) = 5163.93$ | sum of log ratios   |
| $n/S = 4392/5163.93 = 0.8505$                  |                     |
| $\hat{\alpha} = 1 + 0.8505 = 1.851$            | estimate            |

(b) What is the  $\lambda$  if the distribution is exponential?

$\lambda = 0.1606$ . MLE  $\hat{\lambda} = 1/\bar{x} = 1/6.227$ .

|                                    |              |
|------------------------------------|--------------|
| $\sum_i x_i = 27348.7$             | sum of votes |
| $\bar{x} = 27348.7/4392 = 6.227$   | mean         |
| $\hat{\lambda} = 1/6.227 = 0.1606$ | estimate     |

(c) What is  $[a, b]$  if the distribution is uniform?

$[a, b] = [1.9, 8.5]$ , the minimum and maximum votes (MLE).

|                              |              |
|------------------------------|--------------|
| $\hat{a} = \min_i x_i = 1.9$ | lowest vote  |
| $\hat{b} = \max_i x_i = 8.5$ | highest vote |

(d) What is  $\mu$  and  $\sigma$  if the distribution is normal?

$\mu = 6.227$ ,  $\sigma = 0.893$  (sample mean and sample standard deviation with  $n - 1$ ).

|                                        |                                 |
|----------------------------------------|---------------------------------|
| $\hat{\mu} = \bar{x} = 6.227$          | mean, as in the exponential fit |
| $\sum_i (x_i - \bar{x})^2 = 3503.28$   | sum of squared deviations       |
| $3503.28/(4392 - 1) = 0.7978$          | sample variance                 |
| $\hat{\sigma} = \sqrt{0.7978} = 0.893$ | standard deviation              |

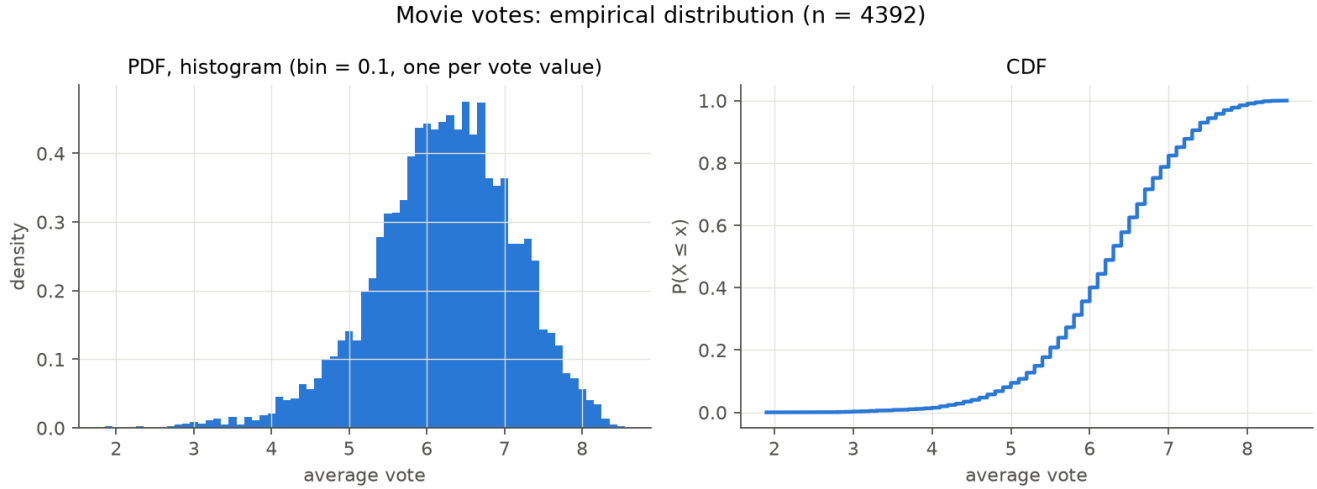
(These can be CDF, PDF, or QQplot)

For (f) to (i):

- **Samples.**  $n = 4392$  random values drawn from each fitted model (same seed).
- **Colours.** Data in blue, model sample in orange.
- **D.** Two-sample Kolmogorov–Smirnov distance: the largest vertical gap between the two CDFs (0 = identical).



(e) Screen shot of the visualization of distribution of data

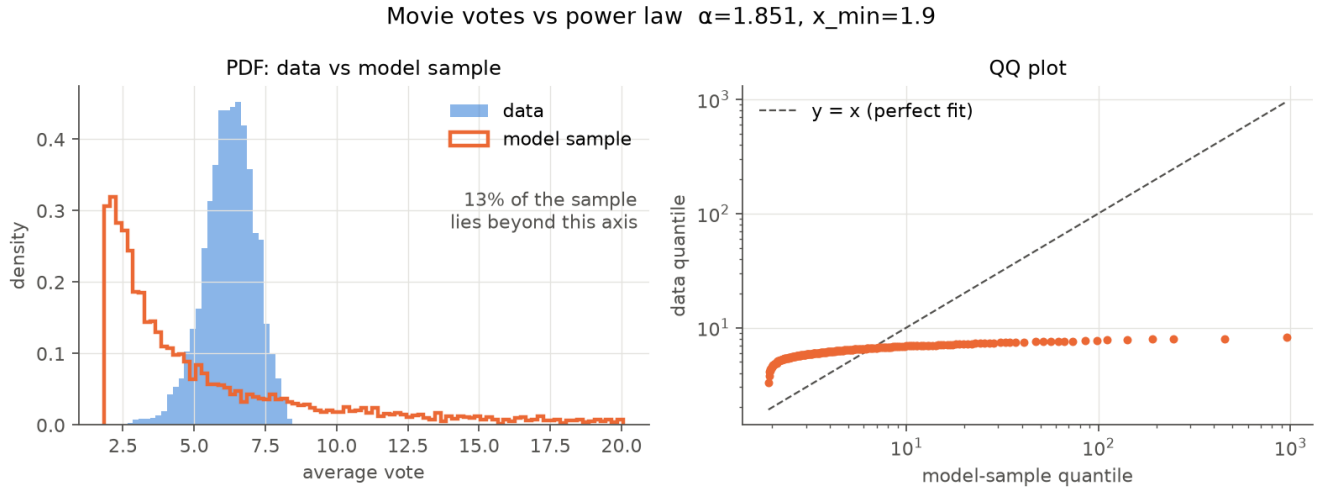


**Figure 6:** Movie votes: empirical distribution ( $n = 4392$ ).

**PDF (left).** Histogram with one bin per possible vote (votes are rounded to 0.1). One peak near 6.3 and a nearly symmetric shape; the left tail is longer (skewness  $-0.48$ ): a few movies are rated 2 to 4, and none are above 8.5.

**CDF (right).** An S-shaped curve centred near 6.3 (median 6.3, mean 6.23). The small steps come from the 0.1 rounding.

(f) Screen shot of the visualization of distribution of data if it was power law

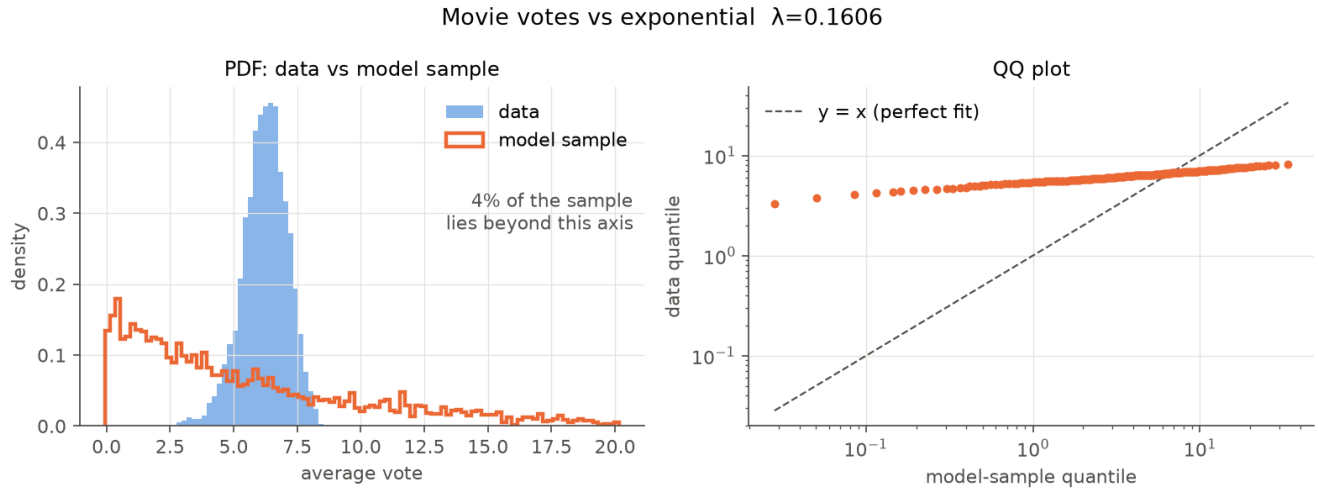


**Figure 7:** Movie votes vs. power law ( $\alpha = 1.851$ ,  $x_{\min} = 1.9$ ).  $D = 0.485$ .

**PDF (left).** The model puts its highest density at the lowest vote (1.9), where the data has almost nothing, and then a long tail. 13% of its sample lies above 20, which is not a possible rating.

**QQ plot (right).** Log axes, because the sample goes up to about 1000. At the top votes the points run almost horizontally to the right: the model's high quantiles are hundreds, the data's stop at 8.5.

(g) Screen shot of the visualization of distribution of data if it was exponential

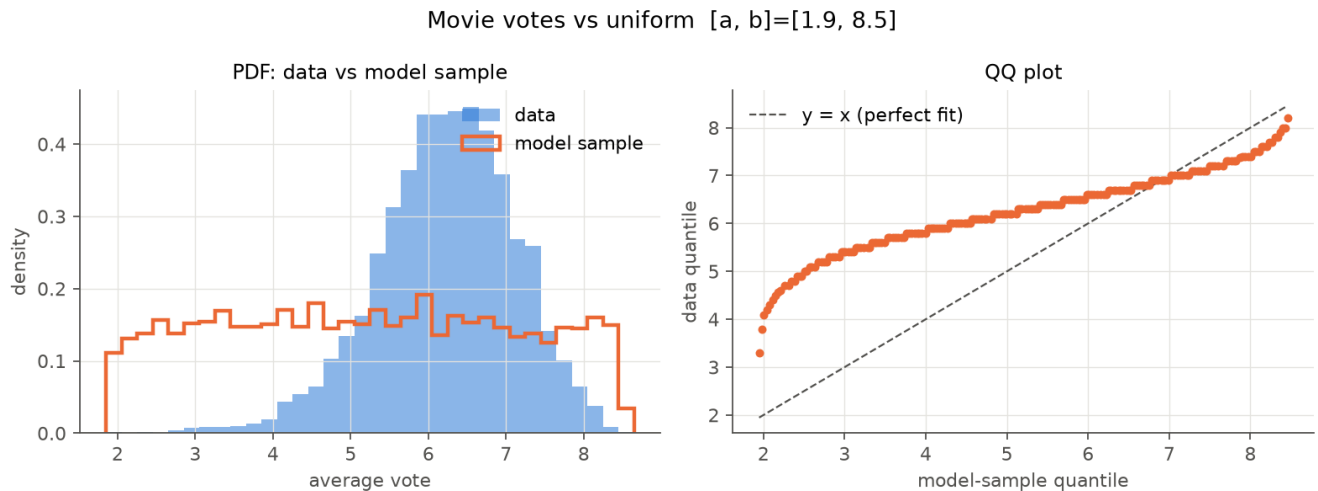


**Figure 8:** Movie votes vs. exponential ( $\lambda = 0.1606$ ).  $D = 0.490$ .

**PDF (left).** The model peaks at zero and has a long right tail; 4% of its sample is above 20. The data is a narrow hump between 4 and 8.5.

**QQ plot (right).** The points cross  $y = x$  at a shallow angle: the model's spread is far larger than the data's, with low quantiles near 0 and high quantiles above 20.

(h) Screen shot of the visualization of distribution of data if it was uniform

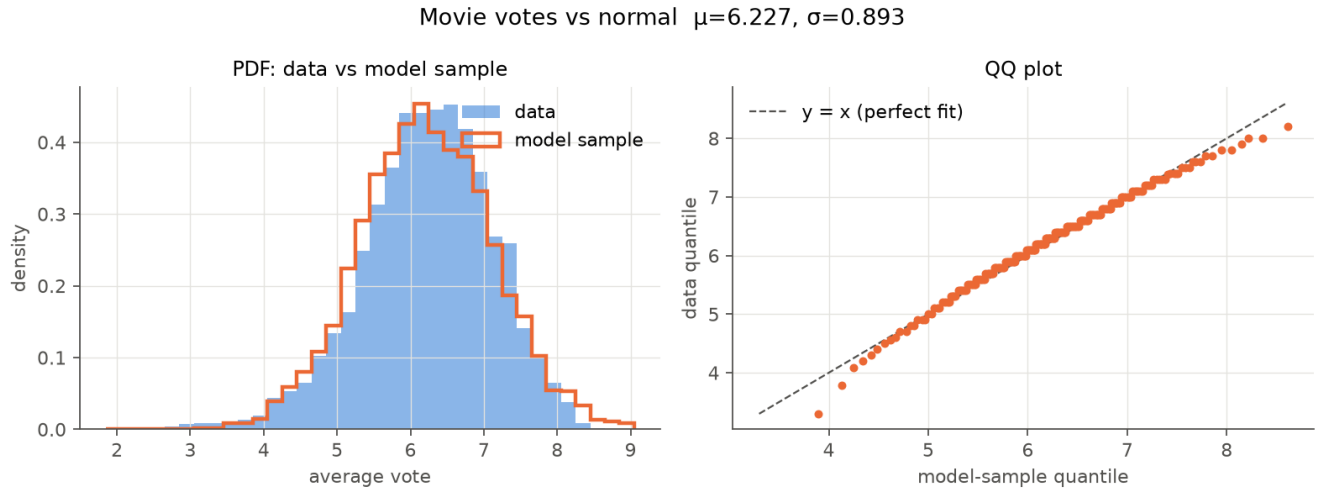


**Figure 9:** Movie votes vs. uniform on  $[1.9, 8.5]$ .  $D = 0.394$ .

**PDF (left).** The model is flat across  $[1.9, 8.5]$ , but the data is concentrated between 5 and 7.5.

**QQ plot (right).** The points lie flat across the middle, above  $y = x$  on the left and below it on the right: the model's low quantiles are too low and its high quantiles too high, because it spreads the votes evenly instead of piling them in the middle.

(i) Screen shot of the visualization of distribution of data if it was normal



**Figure 10:** Movie votes vs. normal ( $\mu = 6.227, \sigma = 0.893$ ).  $D = 0.055$ .

**PDF (left).** The sample traces the same bell as the data, with the peak in the same place.

**QQ plot (right).** The points lie on  $y = x$  from about 4.5 to 7.5. The only departures are at the ends: the data's lowest votes are lower than the model predicts (longer left tail), and the data stops at 8.5 while the model continues.  $D = 0.055$  is by far the smallest.

(j) What is the distribution of data based on your observation? Discuss

|                                        | power law | exponential | uniform | normal       |
|----------------------------------------|-----------|-------------|---------|--------------|
| KS distance $D$ (data vs model sample) | 0.485     | 0.490       | 0.394   | <b>0.055</b> |

**The movie votes follow a normal distribution, approximately.** The data has one peak near the middle of its range and is close to symmetric. The normal model predicts 68.3% of movies within  $\pm 1\sigma$  and 95.4% within  $\pm 2\sigma$ ; the data has 70.1% and 95.6%. In the QQ plot the points lie on the identity line through the bulk of the data, and the KS distance is about one seventh of the next-best model's.

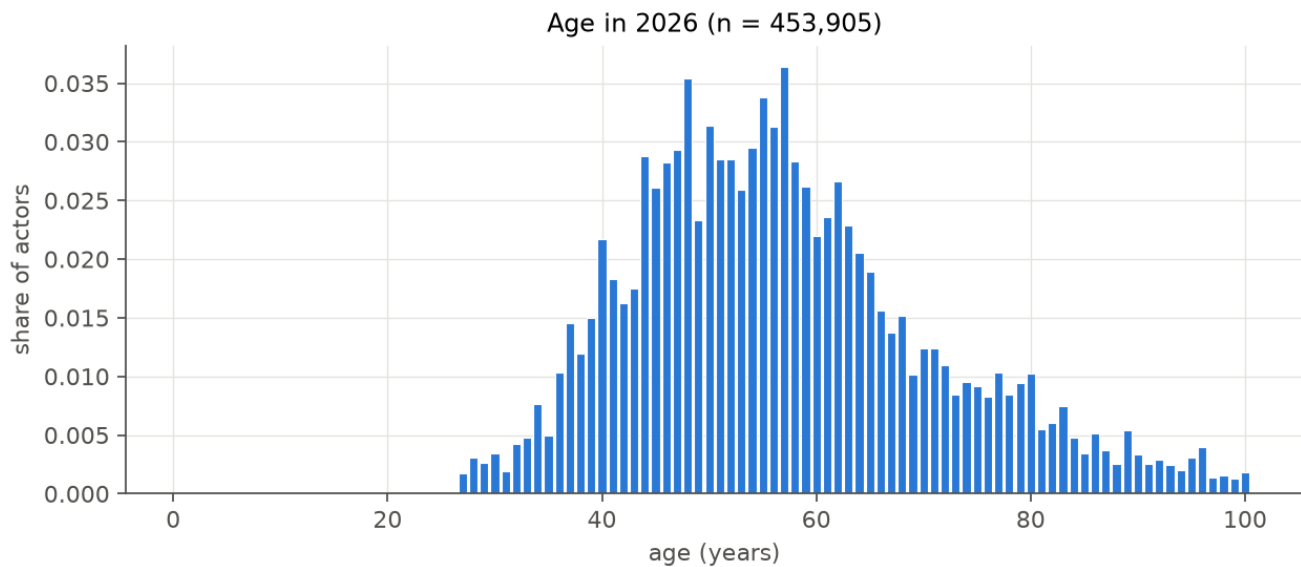
The other three models fail for structural reasons. The power law and the exponential are both monotonically decreasing: their most likely value is the smallest one, so they predict mostly terrible movies and a long tail of ratings above 10. The uniform model ignores the peak altogether.

The fit is good but not exact, for two reasons. First, the data is bounded to  $[1, 10]$  and slightly left-skewed: bad movies can drop further below the average than good movies can rise above it. Second, votes are rounded to one decimal, which gives the QQ plot its small steps. A normal shape is expected because each value is an *average* of many individual votes. By the central limit theorem an average of many independent ratings tends toward a normal distribution, whatever the distribution of each single rating.

## Problem 2.

### - Screenshot of distribution/histogram of the age

How the ages were computed. The birth dates have the form **dd-Mon-yy** (a few use a 4-digit year). Every 2-digit year is read as **19yy**: IMDB-WIKI was collected in 2015, so a year such as 20–24 cannot mean 2020–2024, and the counts per year decrease smoothly from the 1920s down to 00, with no break that would mark a switch to the 2000s. The 129 rows with year 0000 (a missing-value placeholder) were dropped. Per the professor’s instruction (2026-09-23), age is now the year difference **2026 minus birth year** (month/day ignored), kept only if  $1 \leq \text{age} \leq 100$ . Keeping that range leaves **453,905** people (mean 56.87, median 55, range 27 to 100). The minimum of 27 is a result of the **19yy** rule, since the latest possible birth year is 1999 ( $2026 - 1999 = 27$ ). Of the rest, 6,684 rows had age  $> 100$  and were dropped; none had age  $< 1$ .

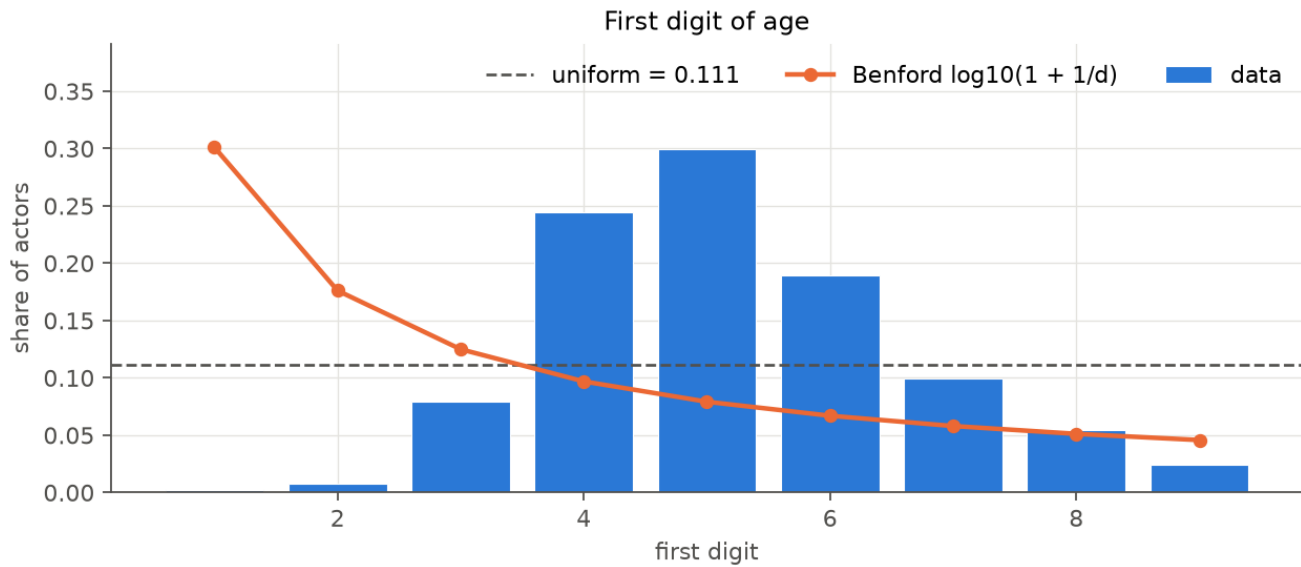


**Figure 11:** Age in 2026 ( $n = 453,905$ ).

**Shape.** One broad hump: ages start at 27, rise through the 30s, peak in the 40s and 50s, and fall off in a long right tail up to 100. Most of the mass is between 36 and 71.

**Jagged bars.** Neighbouring years differ (for example, 48 and 57 stand out). This is year-to-year noise in how many actors were born in a given year, not a pattern in the ages themselves.

- Screenshot of distribution/histogram of the first digit of age



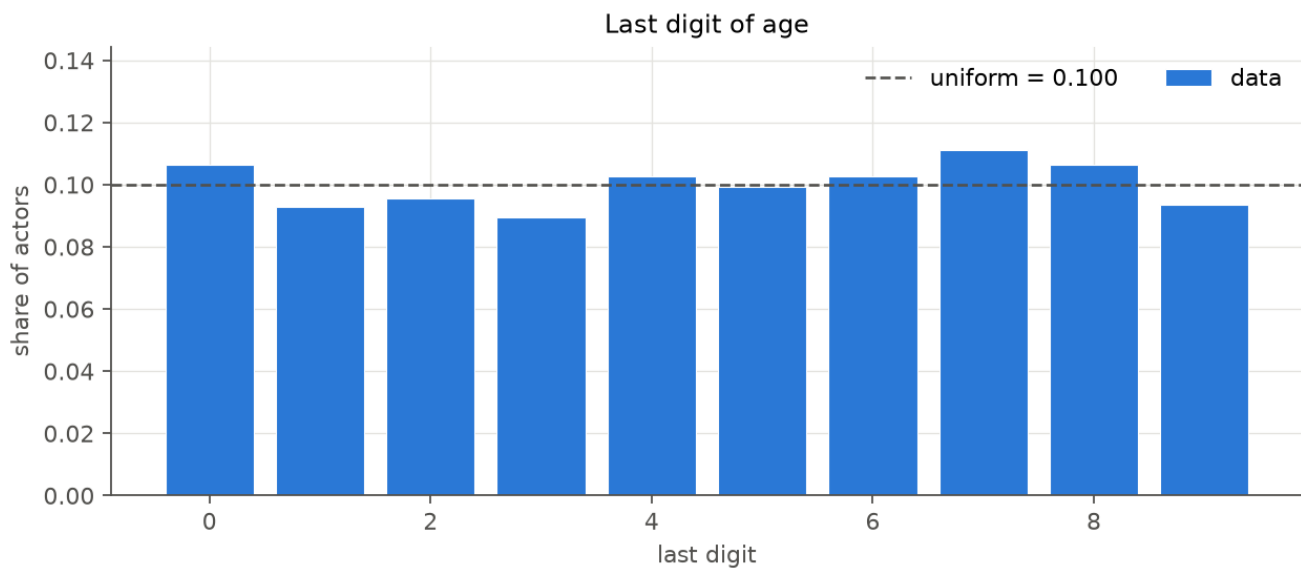
**Figure 12:** First digit of age, with the uniform level  $1/9$  (dashed) and Benford's law (orange).

**Bars.** The shares are 0.2%, 0.7%, 7.9%, 24.5%, 30.0%, 18.9%, 9.9%, 5.4% and 2.5% for digits 1 to 9. They peak at 5 and 4, the same place as the hump in the age histogram.

**Uniform line.** Far from the dashed  $1/9 = 0.111$  level: digits 4 to 6 are well above it, digits 1 to 3 and 8 to 9 well below.

**Benford curve.** Also far from Benford, which predicts 1 as the most common digit (30.1%); here 1 is the rarest.

- Screenshot of distribution/histogram of the last digit of age



**Figure 13:** Last digit of age, with the uniform level  $1/10$  (dashed).

**Bars.** Every digit has a share between 8.95% (digit 3) and 11.10% (digit 7), within 1.10 percentage points of 10%.

**Uniform line.** All ten bars sit close to the dashed 0.100 level, with no digit clearly favoured.

- Discussion (what are those distribution?) Why do you think it is like this?

**The last digit is uniform (approximately); the first digit is not.**

**Last digit.** This is expected. The last digit of an age is its value modulo 10. Every block of ten ages (20–29, 30–39, ...) contains each last digit exactly once. The age distribution changes slowly compared with a step of one year, so within any block of ten the ten ages have roughly equal counts, and summing over the blocks gives each last digit about 10%. The remaining deviations of up to  $\pm 1.1$  points come from year-to-year noise in how many actors were born in a given year (the jagged bars in the age histogram), not from any preference for particular digits.

**First digit.** This is also expected, once the two boundary decades are accounted for separately. For ages 30 to 99, the first digit *is* the decade of life, and grouping by decade reproduces the shape of the age distribution: it peaks at 5 (the 50s, 30.0%) and 4 (the 40s, 24.5%), matching the median of 55. Digit 2 is small (0.7%) not because few actors are in their 20s, but because the  $1 \leq \text{age} \leq 100$  rule only keeps ages 27 to 29 in that decade, three values out of ten, a boundary effect of the reference year rather than a property of the data. Digit 1 is smaller still (0.2%): no age from 10 to 19 survives the rule (the youngest possible age is 27), so the entire digit-1 count comes from the 816 people with age exactly 100 ( $816/453,905 = 0.18\%$ , matching the observed share). For the first digit to be uniform, every decade would need the same number of actors, which working actors clearly are not, and the two extreme digits are further skewed by where the  $[1, 100]$  window happens to cut.

**It is not Benford's law either.** Benford's law (first digit  $d$  with probability  $\log_{10}(1 + 1/d)$ , so 1 is the most common at 30.1%) applies to data that spans several orders of magnitude, such as populations, prices or the airport route counts in Problem 1, where the log of the value is spread out roughly evenly. Age spans less than one order of magnitude (27 to 100) and is concentrated around 55, so its first digit shows where that one hump sits, not Benford's pattern. In short, the last digit is uniform because it throws away where the value is and keeps only its position within a block of ten. The first digit keeps exactly that information, so its distribution follows the shape of the age distribution.

**Tooling.** Python 3 / pandas / numpy / scipy / matplotlib; all estimates computed programmatically, none typed by hand; analysis code, figure code and write-up drafted with Claude Code. Code: `hw2/code/p1.py` and `hw2/code/p2.py`, listed below.

## Code

### Problem 1: hw2/code/p1.py

```
"""
Problem 1: fit power law, exponential, uniform and normal models to
  A. airport routes (routes per airport)
  B. movie votes (average TMDb rating)
and compare each fitted model against the data by drawing random samples from it.

Run from the repo root: uv run python hw2/code/p1.py
Writes figures to hw2/figures/ and prints every estimate used in the report.
"""

from pathlib import Path

import matplotlib

matplotlib.use("Agg") # headless: write PNGs only, no GUI backend needed
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import ks_2samp

HW_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = HW_DIR / "00_doc"
FIG_DIR = HW_DIR / "figures"
SEED = 6330
VOTE_VIEW_MAX = 20.0 # x-limit for movie PDFs; ratings live in [1, 10]

DATA_COLOR = "#2a78d6"
MODEL_COLOR = "#eb6834"
INK = "#52514e"
GRID = "#e4e3df"

plt.rcParams.update({
    "axes.edgecolor": INK, "axes.labelcolor": INK, "xtick.color": INK, "ytick.color": INK,
    "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.6,
    "axes.spines.top": False, "axes.spines.right": False,
    "font.size": 10, "axes.titlesize": 11, "legend.frameon": False,
})

# ----- estimators (pure)

def fit_power_law(x: np.ndarray) -> dict[str, float]:
    """Continuous MLE (Clauset et al. 2009) with  $x_{\min} = \min(x)$ :  $\alpha = 1 + n / \sum \ln(x/x_{\min})$ """
    x_min = float(x.min())
    alpha = 1.0 + len(x) / float(np.log(x / x_min).sum())
    return {"alpha": alpha, "x_min": x_min}

def fit_exponential(x: np.ndarray) -> dict[str, float]:
    """MLE:  $\lambda = 1 / \text{mean}$ """
    return {"lambda": 1.0 / float(x.mean())}

def fit_uniform(x: np.ndarray) -> dict[str, float]:
    """MLE:  $[a, b] = [\min, \max]$ """
    return {"a": float(x.min()), "b": float(x.max())}

def fit_normal(x: np.ndarray) -> dict[str, float]:
```

```

"""mu = sample mean, sigma = sample standard deviation (ddof = 1)."""
return {"mu": float(x.mean()), "sigma": float(x.std(ddof=1))}

def sample_models(x: np.ndarray, rng: np.random.Generator) -> dict[str, tuple[str, np.ndarray]]:
    """Draw len(x) values from each fitted model; returns {key: (label, sample)}."""
    n = len(x)
    pl, ex, un, no = fit_power_law(x), fit_exponential(x), fit_uniform(x), fit_normal(x)
    # Inverse-CDF sampling of the Pareto form:  $x = x_{\min} * (1 - u)^{-1 / (\alpha - 1)}$ .
    pl_sample = pl["x_min"] * (1.0 - rng.random(n)) ** (-1.0 / (pl["alpha"] - 1.0))
    return {
        "powerlaw": (f"power law  $\alpha$ ={pl['alpha']:.3f},  $x_{\min}$ ={pl['x_min']:g}", pl_sample),
        "exponential": (f"exponential  $\lambda$ ={ex['lambda']:.4f}",
                        rng.exponential(1.0 / ex["lambda"], n)),
        "uniform": (f"uniform [a, b]=[{un['a']:g}, {un['b']:g}]",
                    rng.uniform(un["a"], un["b"], n)),
        "normal": (f"normal  $\mu$ ={no['mu']:.3f},  $\sigma$ ={no['sigma']:.3f}",
                   rng.normal(no["mu"], no["sigma"], n)),
    }

# ----- plotting

def ccdf(x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """P(X >= x) at each sorted value - a power law is a straight line on log-log."""
    xs = np.sort(x)
    return xs, 1.0 - np.arange(len(xs)) / len(xs)

def ecdf(x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    xs = np.sort(x)
    return xs, np.arange(1, len(xs) + 1) / len(xs)

def qq_panel(ax: plt.Axes, data: np.ndarray, sample: np.ndarray, log: bool) -> None:
    q = np.linspace(0.005, 0.995, 199)
    dq, sq = np.quantile(data, q), np.quantile(sample, q)
    if log:
        keep = sq > 0 # normal / exponential samples can go <= 0; log axes cannot show them
        dq, sq = dq[keep], sq[keep]
        ax.set_xscale("log")
        ax.set_yscale("log")
    # Usual convention (scipy probplot, statsmodels, R qqnorm, lecture 7 slides): the model
    # is the reference on x, the observed data is on y.
    ax.scatter(sq, dq, s=12, color=MODEL_COLOR, zorder=3)
    lo, hi = min(dq.min(), sq.min()), max(dq.max(), sq.max())
    ax.plot([lo, hi], [lo, hi], color=INK, lw=1, ls="--", label="y = x (perfect fit)")
    ax.set(xlabel="model-sample quantile", ylabel="data quantile", title="QQ plot")
    ax.legend(loc="upper left")

def save(fig: plt.Figure, name: str) -> None:
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"{name}.png", dpi=160)
    plt.close(fig)

def plot_airport_data(x: np.ndarray) -> None:
    # Top row on linear axes shows how bunched up the data is; the bottom row on log-log
    # axes spreads that bunch out so the shape and the tail can be read.
    fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(10, 7.2))
    ax1.hist(x, bins=np.arange(0, x.max() + 20, 20), weights=np.full(len(x), 1 / len(x)),

```



```

        color=DATA_COLOR)
ax1.set(xlabel="routes per airport", ylabel="share of airports",
        title="PDF, linear axes (bin = 20 routes)")
ax2.plot(*ecdf(x), color=DATA_COLOR, lw=2)
share_60 = float(np.mean(x <= 60))
ax2.axvline(60, color=INK, lw=1, ls="--")
ax2.text(80, 0.5, f"{share_60:.0%} of airports\nhave  $\leq 60$  routes", color=INK)
ax2.set(xlabel="routes per airport", ylabel="P(X  $\leq x$ )", title="CDF, linear axes")
# Routes are integers, so plot P(X = k) per value instead of log bins (which leave gaps).
values, counts = np.unique(x, return_counts=True)
ax3.scatter(values, counts / len(x), s=12, color=DATA_COLOR)
ax3.set(xscale="log", yscale="log", xlabel="routes per airport (log)",
        ylabel="P(X = k) (log)", title="PDF, log-log (probability mass per value)")
ax4.plot(*ccdf(x), color=DATA_COLOR, lw=2, label="data")
ax4.set(xscale="log", yscale="log", xlabel="routes per airport (log)",
        ylabel="P(X  $\geq x$ ) (log)", title="CCDF, log-log")
fig.suptitle(f"Airport routes: empirical distribution (n = {len(x)})")
save(fig, "pia_0_data")

def log_binned_pdf(v: np.ndarray, bins: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Density per log bin, normalised by the WHOLE sample so values off the bins still count."""
    counts, edges = np.histogram(v, bins=bins)
    density = counts / (len(v) * np.diff(edges))
    centers = np.sqrt(edges[:-1] * edges[1:])
    keep = counts > 0
    return centers[keep], density[keep]

def plot_airport_model(x: np.ndarray, key: str, label: str, sample: np.ndarray) -> None:
    fig, ((axp, ax0), (ax1, ax2)) = plt.subplots(2, 2, figsize=(10, 7.2))
    # PDF on log-log axes with log-width bins: the model sample is continuous, so the
    # per-value mass used for the data-only figure does not apply here.
    bins = np.logspace(0, 6, 37)
    axp.plot(*log_binned_pdf(x, bins), "o-", ms=4, color=DATA_COLOR, label="data")
    axp.plot(*log_binned_pdf(sample, bins), "o-", ms=4, color=MODEL_COLOR,
            label="model sample")
    outside = float(np.mean((sample < bins[0]) | (sample > bins[-1])))
    if outside > 0.01:
        axp.text(0.98, 0.92, f"{outside:.0%} of the sample\nlies outside [1, 1e6]",
                transform=axp.transAxes, ha="right", va="top", color=INK)
    axp.set(xscale="log", yscale="log", xlabel="routes per airport (log)",
            ylabel="density (log)", title="PDF, log-log (log-width bins)")
    axp.legend(loc="lower left")
    # Linear-axis CDF, cut at the data's range; the share of the sample beyond it is stated.
    ax0.plot(*ecdf(x), color=DATA_COLOR, lw=2, label="data")
    ax0.plot(*ecdf(sample), color=MODEL_COLOR, lw=2, label="model sample")
    x_hi = 1.1 * float(x.max())
    ax0.set_xlim(min(0.0, float(np.quantile(sample, 0.001))), x_hi)
    beyond = float(np.mean(sample > x_hi))
    if beyond > 0.001:
        ax0.text(0.98, 0.35, f"{beyond:.1%} of the sample\nlies beyond this axis",
                transform=ax0.transAxes, ha="right", color=INK)
    ax0.set(xlabel="routes per airport", ylabel="P(X  $\leq x$ )", title="CDF, linear axes")
    ax0.legend(loc="lower right")
    ax1.plot(*ccdf(x), color=DATA_COLOR, lw=2, label="data")
    # CCDF over the FULL sample, then hide x <= 0 (log axis); filtering first would renormalise.
    xs, ps = ccdf(sample)
    ax1.plot(xs[xs > 0], ps[xs > 0], color=MODEL_COLOR, lw=2, label="model sample")
    ax1.set(xscale="log", yscale="log", xlabel="routes per airport (log)",
            ylabel="P(X  $\geq x$ ) (log)", title="CCDF: data vs model sample")
    ax1.legend(loc="lower left")

```

```

qq_panel(ax2, x, sample, log=True)
fig.suptitle(f"Airport routes vs {label}")
save(fig, f"p1a_{key}")

def plot_movie_data(x: np.ndarray) -> None:
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
    ax1.hist(x, bins=np.arange(1.85, 8.65, 0.1), density=True, color=DATA_COLOR)
    ax1.set(xlabel="average vote", ylabel="density",
            title="PDF, histogram (bin = 0.1, one per vote value)")
    ax2.plot(*ecdf(x), color=DATA_COLOR, lw=2)
    ax2.set(xlabel="average vote", ylabel="P(X ≤ x)", title="CDF")
    fig.suptitle(f"Movie votes: empirical distribution (n = {len(x)})")
    save(fig, "p1b_0_data")

def plot_movie_model(x: np.ndarray, key: str, label: str, sample: np.ndarray) -> None:
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
    # A power-law sample reaches the thousands; clip the PDF view and say how much is hidden.
    lo = min(x.min(), np.quantile(sample, 0.001))
    hi = min(max(x.max(), np.quantile(sample, 0.999)), VOTE_VIEW_MAX)
    # Votes sit on a 0.1 grid; bin edges between grid points stop bins catching 1 or 2 values.
    bins = np.arange(np.floor(lo * 10) / 10 - 0.05, hi + 0.2, 0.2)
    width = bins[1] - bins[0]
    # Weights (not density=True) keep each curve normalised to its whole sample, even if clipped.
    ax1.hist(x, bins=bins, weights=np.full(len(x), 1 / (len(x) * width)), color=DATA_COLOR,
            alpha=0.55, label="data")
    ax1.hist(sample, bins=bins, weights=np.full(len(sample), 1 / (len(sample) * width)),
            histtype="step", color=MODEL_COLOR, lw=2, label="model sample")
    hidden = float(np.mean((sample < bins[0]) | (sample > bins[-1])))
    if hidden > 0.01:
        ax1.text(0.98, 0.58, f"{hidden:.0%} of the sample\nlies beyond this axis",
                transform=ax1.transAxes, ha="right", color=INK)
    ax1.set(xlabel="average vote", ylabel="density", title="PDF: data vs model sample")
    ax1.legend(loc="upper right")
    qq_panel(ax2, x, sample, log=hi == VOTE_VIEW_MAX)
    fig.suptitle(f"Movie votes vs {label}")
    save(fig, f"p1b_{key}")

# ----- main

def report(name: str, x: np.ndarray) -> None:
    print(f"\n== {name} (n = {len(x)}, mean = {x.mean():.4f}, median = {np.median(x):.4f})")
    for fit in (fit_power_law, fit_exponential, fit_uniform, fit_normal):
        print(f" {fit.__name__:16s} {fit(x)}")

def main() -> None:
    FIG_DIR.mkdir(exist_ok=True)
    rng = np.random.default_rng(SEED)
    airports = pd.read_csv(DATA_DIR / "01_airport_routes.csv")["NumberOfRoutes"].to_numpy(float)
    movies = pd.read_csv(DATA_DIR / "01_movie_votes.csv")["AverageVote"].to_numpy(float)

    report("airport routes", airports)
    plot_airport_data(airports)
    for i, (key, (label, sample)) in enumerate(sample_models(airports, rng).items(), start=1):
        print(f" KS vs {key:12s} D = {ks_2samp(airports, sample).statistic:.3f}")
        plot_airport_model(airports, f"{i}_{key}", label, sample)

    report("movie votes", movies)
    plot_movie_data(movies)

```

```

for i, (key, (label, sample)) in enumerate(sample_models(movies, rng).items(), start=1):
    print(f" KS vs {key:12s} D = {ks_2samp(movies, sample).statistic:.3f}")
    plot_movie_model(movies, f"{i}_{key}", label, sample)

if __name__ == "__main__":
    main()

```

## Problem 2: hw2/code/p2.py

```

"""
Problem 2: age, first digit and last digit of IMDB-WIKI birth dates.

Parsing decisions (checked against the raw year counts, not assumed):
- Dates are "dd-Mon-yy" or "dd-Mon-yyyy". Every 2-digit year is read as 19yy: IMDB-WIKI was
  crawled in 2015, so "20"- "24" cannot be 2020-2024, and the counts fall smoothly from the
  1920s down to "00" with no break that would mark a century switch.
- Year "0000" is a missing-value placeholder and is dropped.
- Age = REFERENCE_YEAR - birth year (year difference only, month/day ignored), kept only if
  1 <= age <= 100. Per owner/professor direction (2026-09-23): 2026 is the reference year,
  and ages outside [1, 100] are dropped as unreasonable.

Run from the repo root: uv run python hw2/code/p2.py
"""

from pathlib import Path

import matplotlib

matplotlib.use("Agg") # headless: write PNGs only, no GUI backend needed
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

HW_DIR = Path(__file__).resolve().parent.parent
DATA_PATH = HW_DIR / "00_doc" / "01_DoB.txt"
FIG_DIR = HW_DIR / "figures"
REFERENCE_YEAR = 2026

DATA_COLOR = "#2a78d6"
BENFORD_COLOR = "#eb6834"
INK = "#52514e"
GRID = "#e4e3df"

plt.rcParams.update({
    "axes.edgecolor": INK, "axes.labelcolor": INK, "xtick.color": INK, "ytick.color": INK,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRID, "grid.linewidth": 0.6,
    "axes.spines.top": False, "axes.spines.right": False,
    "font.size": 10, "axes.titlesize": 11, "legend.frameon": False,
})

def parse_birth_dates(raw: pd.Series) -> pd.Series:
    """dd-Mon-yy / dd-Mon-yyyy -> Timestamp; 2-digit years -> 19yy; year 0000 -> NaT."""
    parts = raw.str.strip().str.split("-", expand=True)
    year = parts[2].astype(int)
    year = year.where(parts[2].str.len() == 4, 1900 + year)
    year = year.where(year > 0)
    text = parts[0] + "-" + parts[1] + "-" + year.astype("Int64").astype(str)
    return pd.to_datetime(text, format="%d-%b-%Y", errors="coerce")

def compute_ages(birth: pd.Series, reference_year: int) -> pd.Series:

```

```

"""Year difference only (month/day ignored), restricted to 1-100."""
age = (reference_year - birth.dt.year).dropna().astype(int)
return age[(age >= 1) & (age <= 100)]

def share_per_value(values: pd.Series, support: range) -> pd.Series:
    return values.value_counts(normalize=True).reindex(support, fill_value=0.0)

def plot_bars(share: pd.Series, uniform: float | None, title: str, xlabel: str, name: str,
              benford: pd.Series | None = None) -> None:
    fig, ax = plt.subplots(figsize=(8, 3.6))
    ax.bar(share.index, share.to_numpy(), width=0.8, color=DATA_COLOR, edgecolor="white", lw=0.5,
          label="data")
    if uniform is not None:
        ax.axhline(uniform, color=INK, ls="--", lw=1.2, label=f"uniform = {uniform:.3f}")
    if benford is not None:
        ax.plot(benford.index, benford.to_numpy(), color=BENFORD_COLOR, marker="o", ms=5, lw=2,
              label="Benford log10(1 + 1/d)")
    if uniform is not None or benford is not None:
        ax.set_ylim(top=max(share.max(), 0.0 if benford is None else benford.max()) * 1.3)
        ax.legend(loc="upper right", ncol=3) # headroom + one row keeps the legend off the bars
    ax.set(title=title, xlabel=xlabel, ylabel="share of actors")
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"{name}.png", dpi=160)
    plt.close(fig)

def main() -> None:
    FIG_DIR.mkdir(exist_ok=True)
    raw = pd.read_csv(DATA_PATH, header=None, names=["dob"], dtype=str)["dob"]
    birth = parse_birth_dates(raw)
    print(f"rows = {len(raw)}, unparseable/placeholder = {birth.isna().sum()}")
    print(f"birth years: min = {birth.dt.year.min():.0f}, max = {birth.dt.year.max():.0f}")

    raw_age = (REFERENCE_YEAR - birth.dt.year).dropna().astype(int)
    age = compute_ages(birth, REFERENCE_YEAR)
    first = age.astype(str).str[0].astype(int)
    last = age % 10
    print(f"ages kept (1-100) = {len(age)}, mean = {age.mean():.2f}, median = {age.median():.0f}, "
          f"min = {age.min()}, max = {age.max()}")
    print(f"dropped by reason: unparseable/placeholder = {birth.isna().sum()}, "
          f"age < 1 = {int((raw_age < 1).sum())}, age > 100 = {int((raw_age > 100).sum())}")

    print("boundary counts: "
          f"age in [1, 10] = {int(((age >= 1) & (age <= 10)).sum())}, "
          f"age == 100 = {int((age == 100).sum())}, "
          f"age > 100 = {int((raw_age > 100).sum())}, "
          f"age < 1 = {int((raw_age < 1).sum())}")

    first_share = share_per_value(first, range(1, 10))
    last_share = share_per_value(last, range(10))
    print("first digit share:", first_share.round(4).to_dict())
    print("last digit share: ", last_share.round(4).to_dict())
    benford = pd.Series({d: float(np.log10(1 + 1 / d)) for d in range(1, 10)})
    print("Benford reference:", benford.round(4).to_dict())
    print(f"max |last share - 0.1| = {(last_share - 0.1).abs().max():.4f}")

    plot_bars(share_per_value(age, range(1, 101)), None,
              f"Age in {REFERENCE_YEAR} (n = {len(age):,})", "age (years)", "p2_1_age")
    plot_bars(first_share, 1 / 9, "First digit of age", "first digit", "p2_2_first_digit",
              benford=benford)

```

```
plot_bars(last_share, 1 / 10, "Last digit of age", "last digit", "p2_3_last_digit")

if __name__ == "__main__":
    main()
```